CS3014 - Estructuras de Datos Avanzadas

Jerarquía de Memoria II
Semana 10

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

14 de octubre de 2026

Objetivos de la sesión



Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Construir la estructura que alcanza la cota de búsqueda en memoria externa: el B-Tree

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Construir la estructura que alcanza la cota de búsqueda en memoria externa: el B-Tree
- Analizar búsqueda, inserción y eliminación en un B-Tree contando transferencias

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Construir la estructura que alcanza la cota de búsqueda en memoria externa: el B-Tree
- Analizar búsqueda, inserción y eliminación en un B-Tree contando transferencias
- Definir el modelo cache-oblivious: algoritmos que no conocen B ni M

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Construir la estructura que alcanza la cota de búsqueda en memoria externa: el B-Tree
- Analizar búsqueda, inserción y eliminación en un B-Tree contando transferencias
- Definir el modelo cache-oblivious: algoritmos que no conocen B ni M
- Entender por qué el orden en que se guarda un árbol en memoria decide cuántas transferencias cuesta recorrerlo

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Lo que quedó pendiente al estudiar el modelo de memoria externa

Buscar entre N elementos necesita $\Omega(\log_{B+1} N)$ transferencias, pero la búsqueda binaria usa $\lg(N/B)$.

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Lo que quedó pendiente al estudiar el modelo de memoria externa

Buscar entre N elementos necesita $\Omega(\log_{B+1} N)$ transferencias, pero la búsqueda binaria usa $\lg(N/B)$.

¿Cómo se aprovecha un bloque entero en cada paso de una búsqueda?

Un bloque trae B elementos. La búsqueda binaria mira uno y descarta la mitad; lo ideal sería mirar los B y descartar casi todo.

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Lo que quedó pendiente al estudiar el modelo de memoria externa

Buscar entre N elementos necesita $\Omega(\log_{B+1} N)$ transferencias, pero la búsqueda binaria usa $\lg(N/B)$.

¿Cómo se aprovecha un bloque entero en cada paso de una búsqueda?

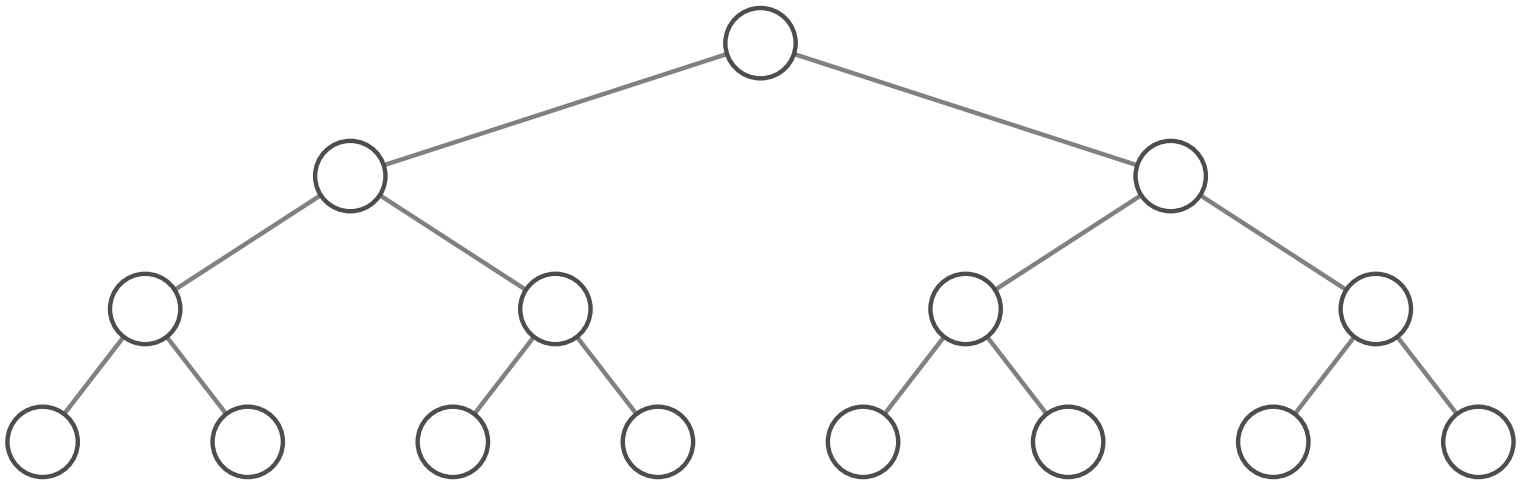
Un bloque trae B elementos. La búsqueda binaria mira uno y descarta la mitad; lo ideal sería mirar los B y descartar casi todo.

Hoy, dos respuestas

Una estructura diseñada **sabiendo** cuánto vale B, y un modelo en el que se nos prohíbe saberlo.

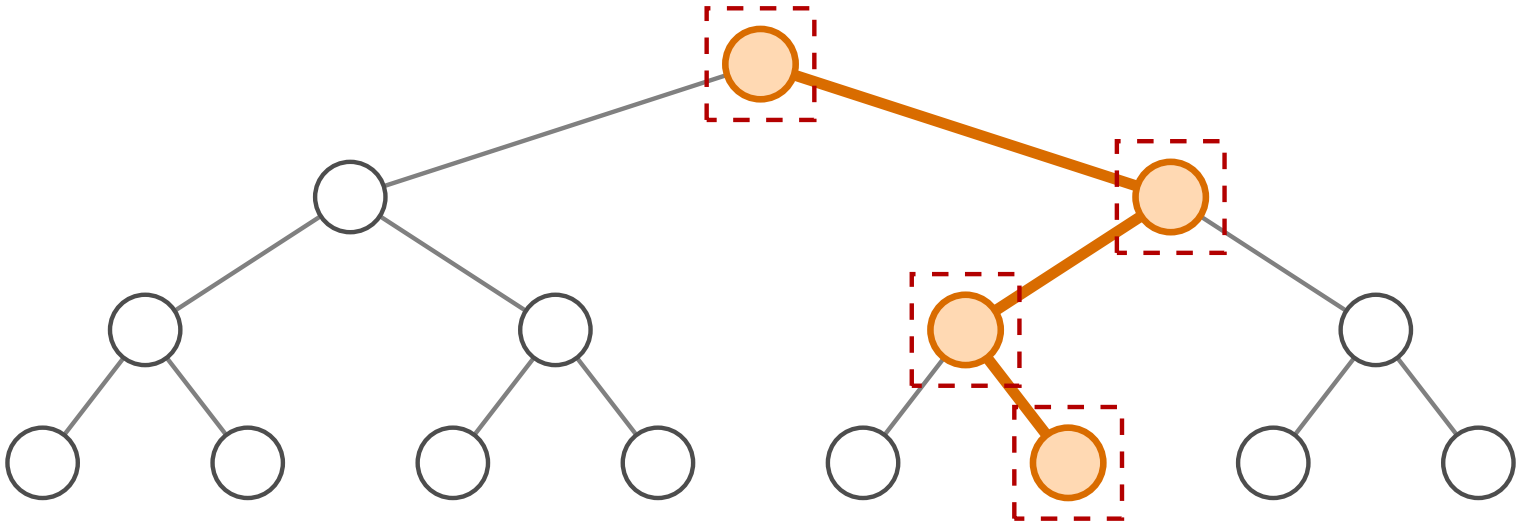
B-Trees

Un árbol binario de búsqueda, en disco



Cada nodo donde caiga

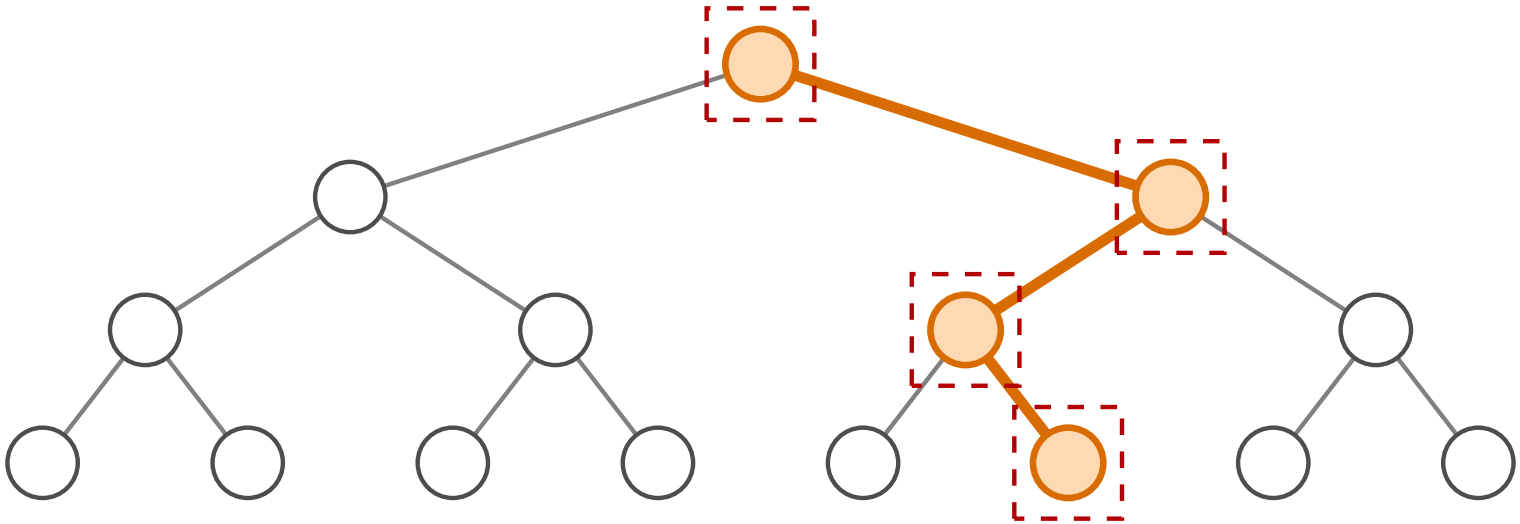
Un árbol binario de búsqueda, en disco



Cada nodo donde caiga

Si los nodos están repartidos por el disco, cada paso de la búsqueda trae un bloque nuevo para leer **una** clave.

Un árbol binario de búsqueda, en disco



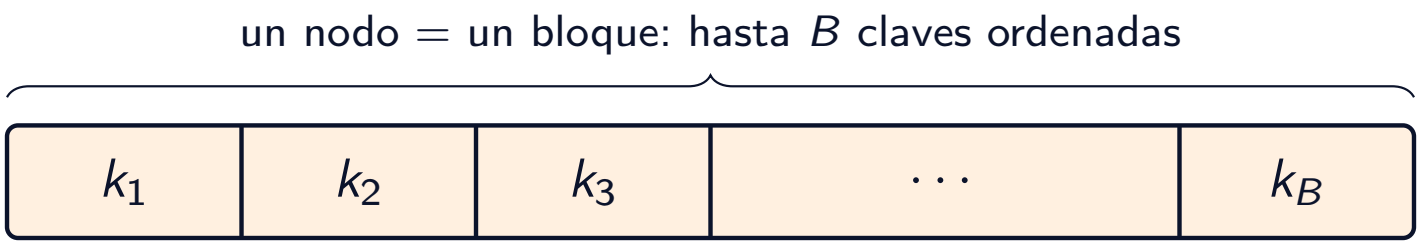
Cada nodo donde caiga

Si los nodos están repartidos por el disco, cada paso de la búsqueda trae un bloque nuevo para leer **una** clave.

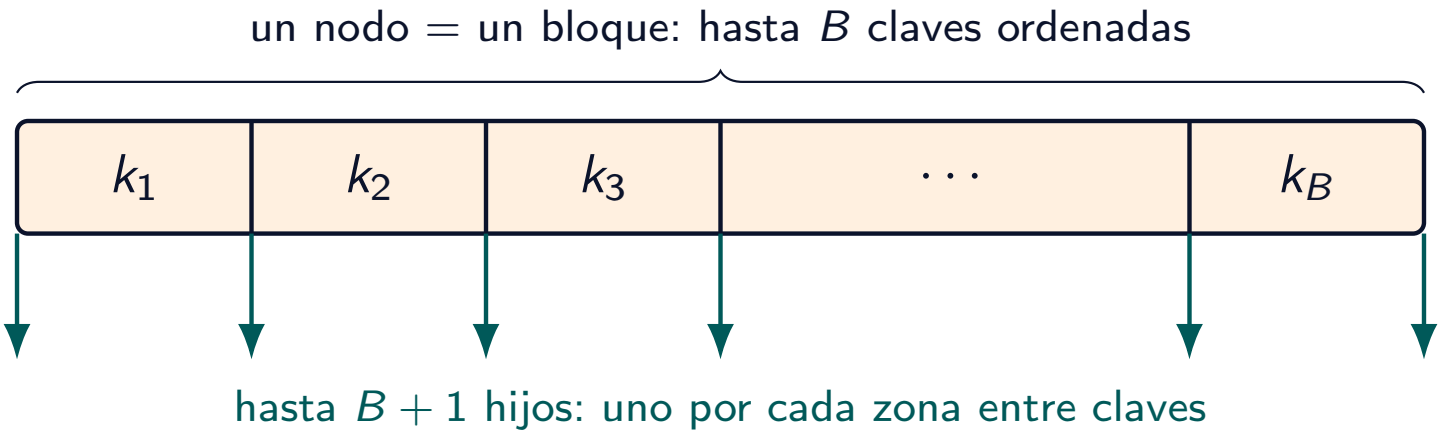
Un árbol balanceado cuesta $\Theta(\lg N)$ transferencias por búsqueda: peor aún que la búsqueda binaria sobre un arreglo.

La idea: nodos del tamaño de un bloque

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA



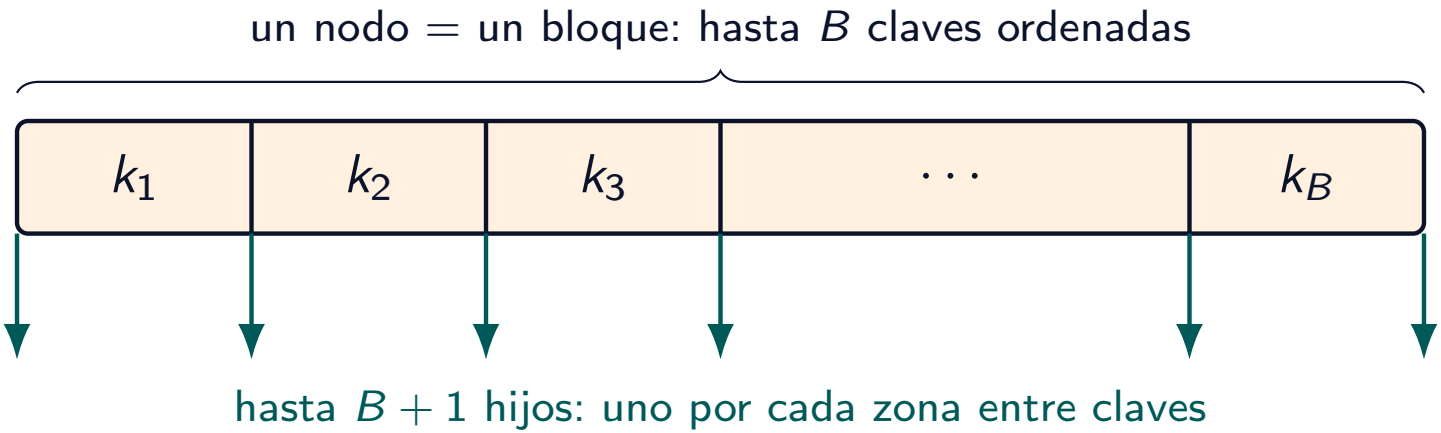
La idea: nodos del tamaño de un bloque



Un nodo con muchas claves

Traer el nodo cuesta una transferencia. Ubicar la clave buscada entre las B del nodo es trabajo en memoria: gratis.

La idea: nodos del tamaño de un bloque



Un nodo con muchas claves

Traer el nodo cuesta una transferencia. Ubicar la clave buscada entre las B del nodo es trabajo en memoria: gratis.

Una sola transferencia decide entre $B+1$ caminos, no entre dos. Es exactamente lo que pedía la cota inferior: $\lg(B+1)$ bits por bloque.

B-Tree

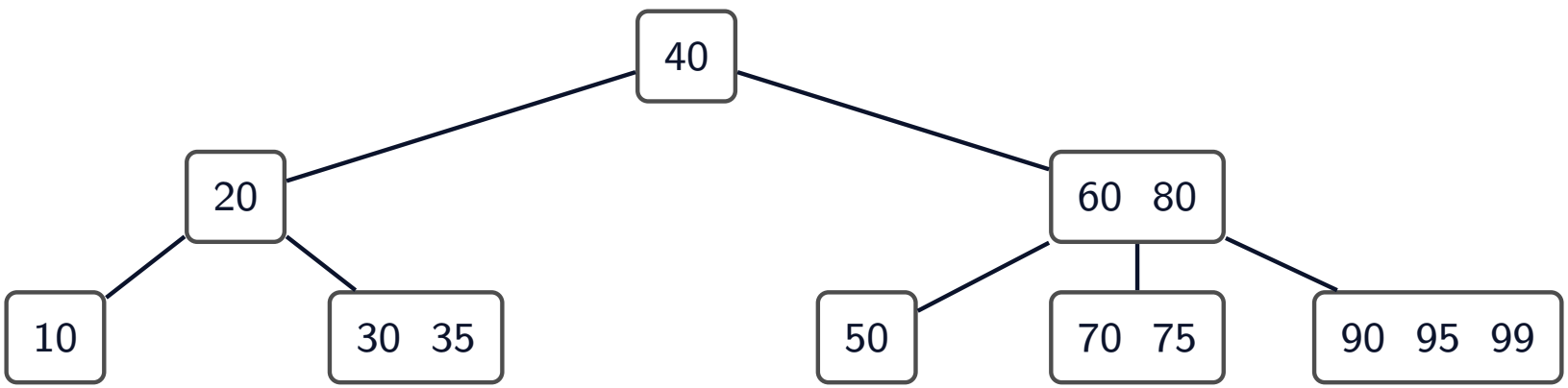
UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Un árbol de búsqueda en el que:

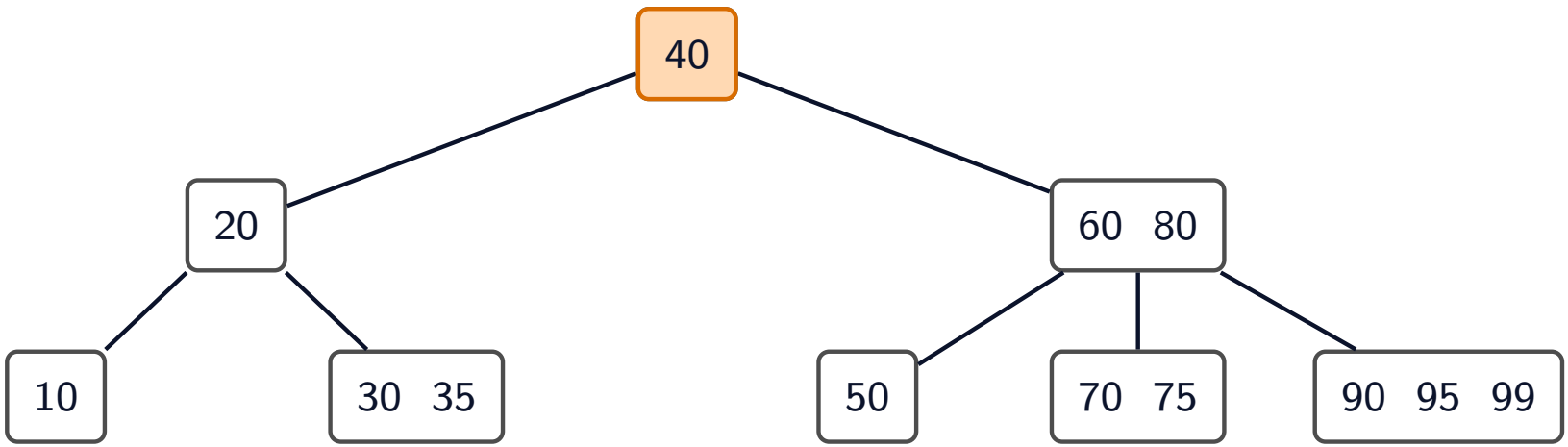
- cada nodo guarda a lo más B claves ordenadas, y un nodo interno con k claves tiene $k+1$ hijos;
- todo nodo, salvo la raíz, tiene al menos $\lceil (B+1)/2 \rceil$ hijos (o esa cantidad de claves menos uno, si es hoja);
- todas las hojas están a la **misma** profundidad.

B-Tree



Un B-Tree con $B = 3$: entre 2 y 4 hijos por nodo.

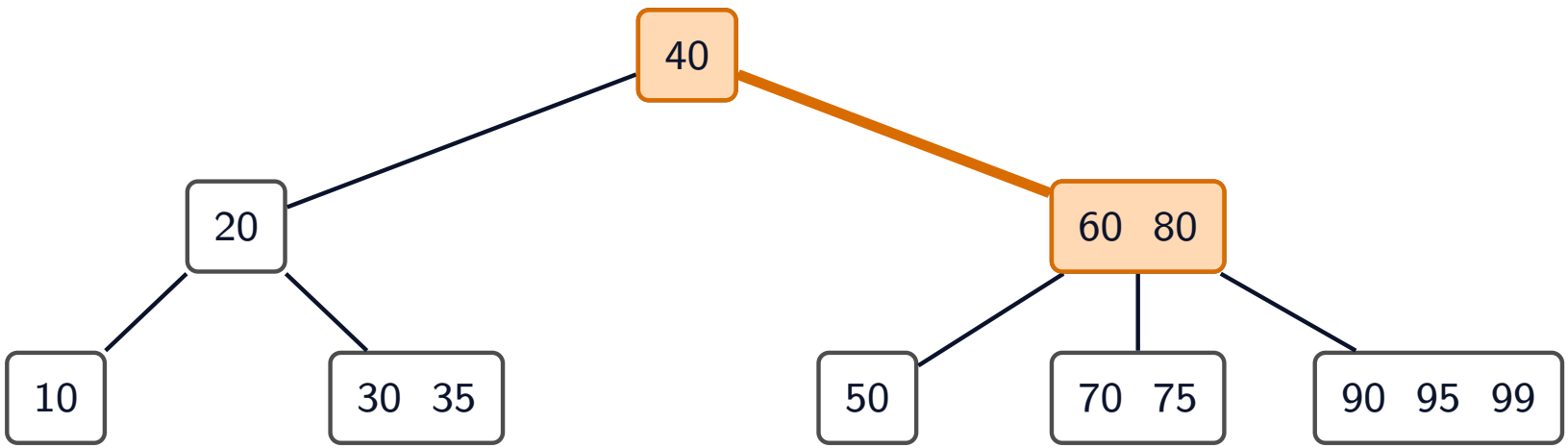
Buscar



Buscar el 75

Se trae la raíz. $75 > 40$: por la derecha.

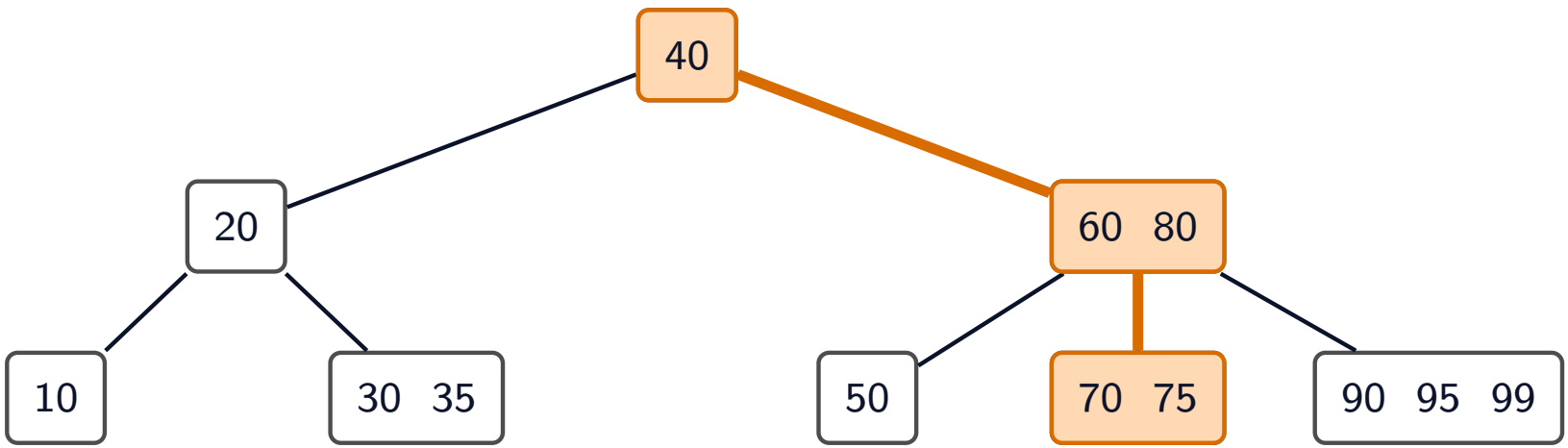
Buscar



Buscar el 75

Se trae el nodo $[60, 80]$. $60 < 75 < 80$: por el hijo del medio.

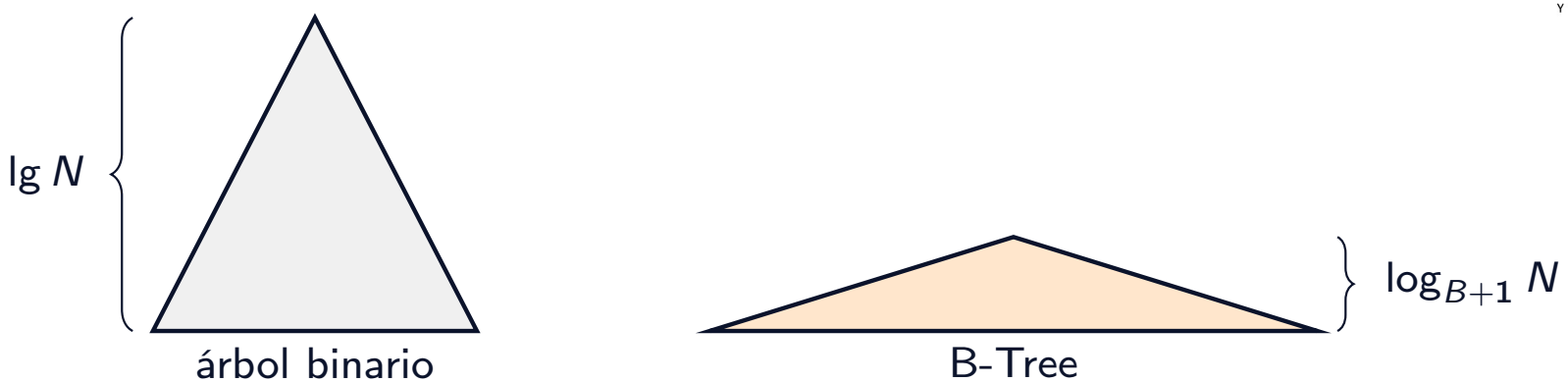
Buscar



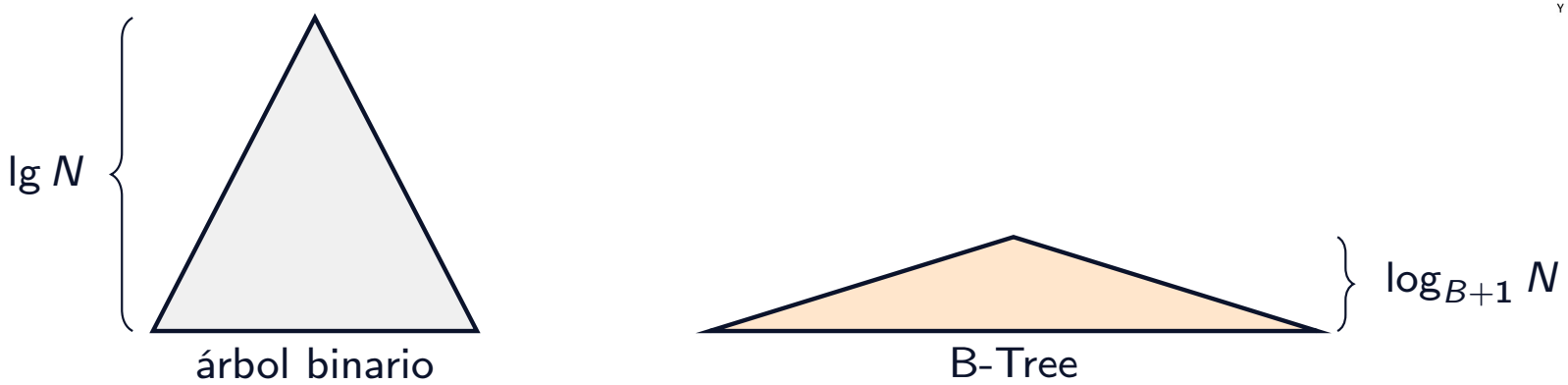
Buscar el 75

Se trae la hoja $[70, 75]$: encontrado. Tres transferencias, una por nivel.

¿Qué tan alto es un B-Tree?



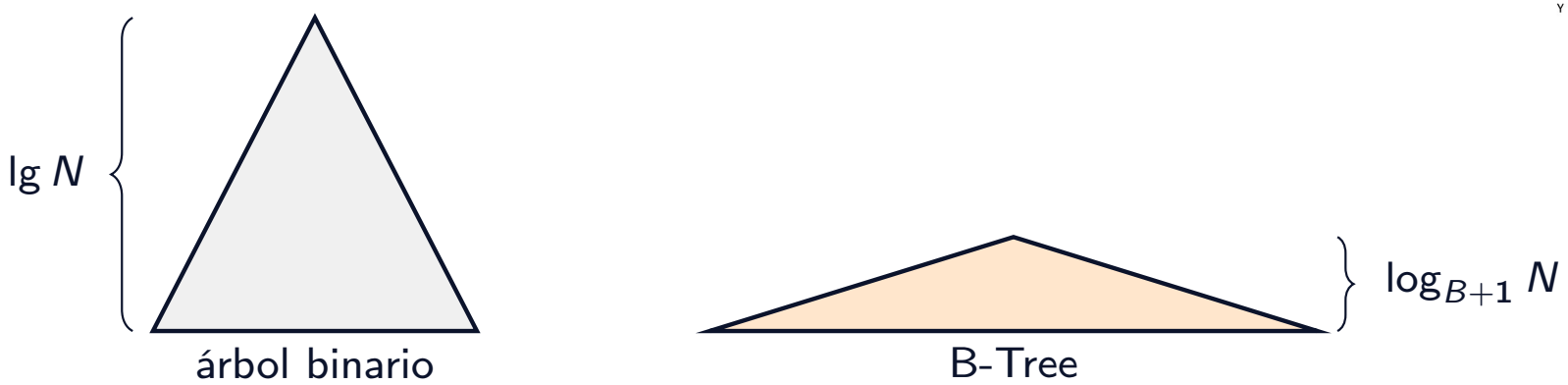
¿Qué tan alto es un B-Tree?



La altura

Cada nodo tiene al menos $\lceil (B+1)/2 \rceil$ hijos: con N claves la altura es $O(\log_{B+1} N)$, unas $\lg(B+1)$ veces menos que la del árbol binario.

¿Qué tan alto es un B-Tree?



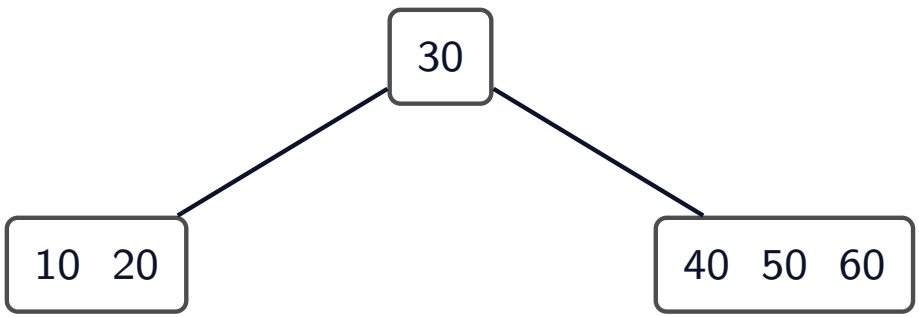
La altura

Cada nodo tiene al menos $\lceil (B+1)/2 \rceil$ hijos: con N claves la altura es $O(\log_{B+1} N)$, unas $\lg(B+1)$ veces menos que la del árbol binario.

Buscar cuesta $\Theta(\log_{B+1} N)$ transferencias: óptimo

Con $N = 10^9$ y $B = 10^3$: 3 o 4 niveles, contra 30 del árbol binario y 20 de la búsqueda binaria.

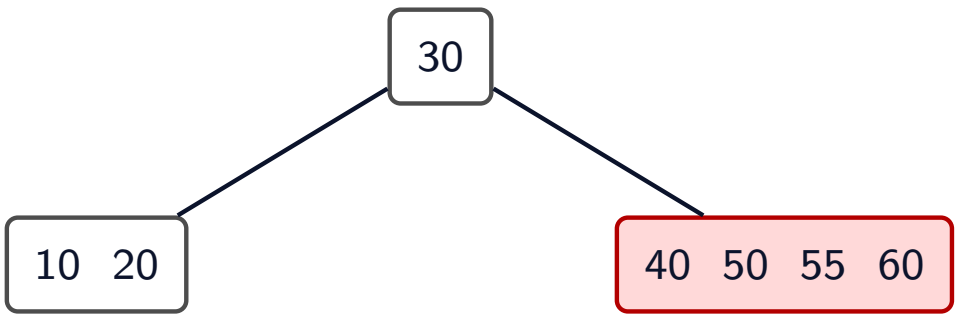
Insertar: partir un nodo lleno



Insertar el 55, con $B = 3$

Se busca la hoja que le corresponde: $[40, 50, 60]$, que ya está llena.

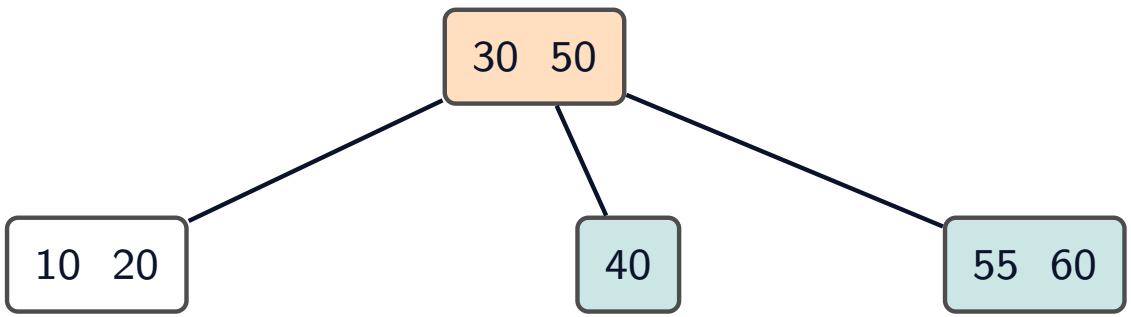
Insertar: partir un nodo lleno



Insertar el 55, con $B = 3$

La hoja quedaría con 4 claves: no cabe en un bloque.

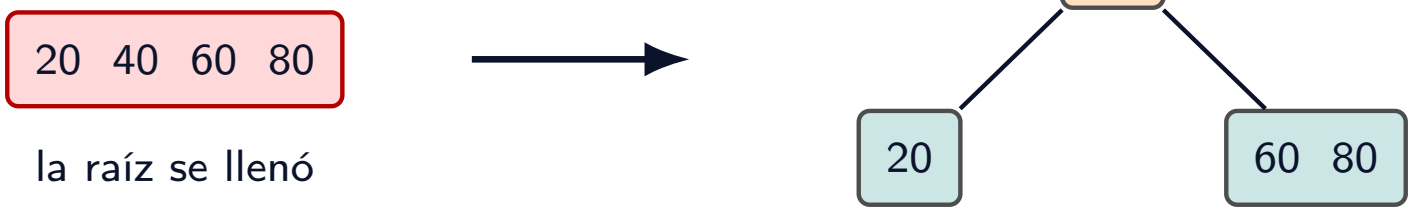
Insertar: partir un nodo lleno



Insertar el 55, con $B = 3$

Se **parte** en dos mitades y la clave del medio, 50, sube al padre. Si el padre también se llena, se parte él, y así hacia arriba.

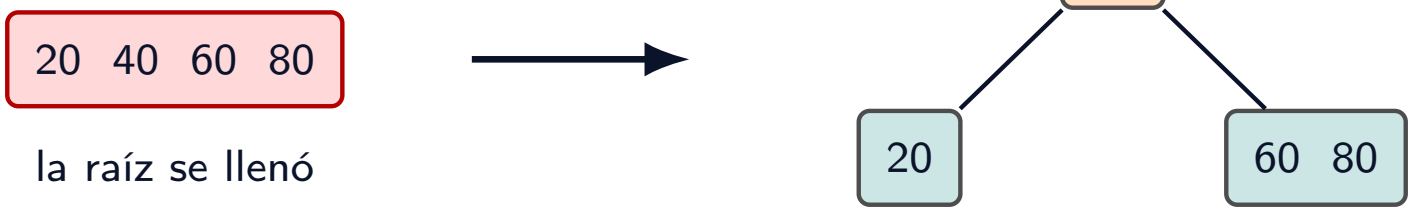
¿Cómo crece un B-Tree?



la raíz se llenó

raíz nueva: un nivel más

¿Cómo crece un B-Tree?



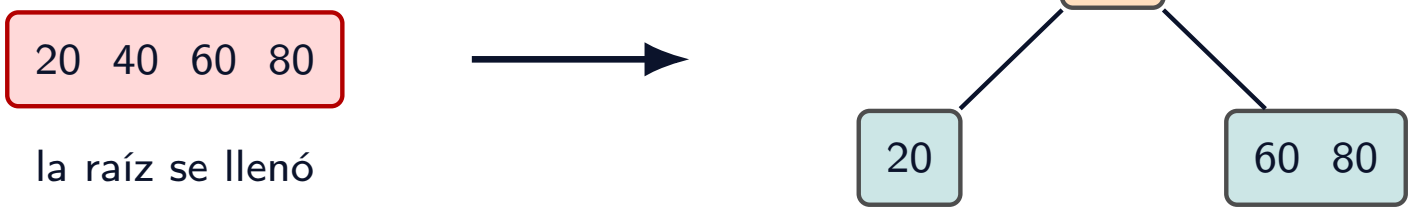
la raíz se llenó

raíz nueva: un nivel más

Hacia arriba, no hacia abajo

Un B-Tree solo gana altura cuando se parte la raíz. Por eso todas las hojas quedan siempre a la misma profundidad, sin rotaciones.

¿Cómo crece un B-Tree?



la raíz se llenó

raíz nueva: un nivel más

Hacia arriba, no hacia abajo

Un B-Tree solo gana altura cuando se parte la raíz. Por eso todas las hojas quedan siempre a la misma profundidad, sin rotaciones.

Eliminar es el proceso inverso

Si un nodo queda con menos claves que el mínimo, toma una prestada de un hermano; si el hermano tampoco tiene de sobra, los dos se **fusionan** y el padre pierde una clave.

El costo de las operaciones

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Operación	Transferencias
Buscar, predecesor, sucesor	$\Theta(\log_{B+1} N)$
Insertar, eliminar	$O(\log_{B+1} N)$
Recorrer k claves consecutivas	$O(\log_{B+1} N + k/B)$

El costo de las operaciones

Operación	Transferencias
Buscar, predecesor, sucesor	$\Theta(\log_{B+1} N)$
Insertar, eliminar	$O(\log_{B+1} N)$
Recorrer k claves consecutivas	$O(\log_{B+1} N + k/B)$

Insertar y eliminar

Una búsqueda hacia abajo y, en el peor caso, una partición o fusión por nivel hacia arriba: un número constante de bloques por nivel.

El costo de las operaciones

Operación	Transferencias
Buscar, predecesor, sucesor	$\Theta(\log_{B+1} N)$
Insertar, eliminar	$O(\log_{B+1} N)$
Recorrer k claves consecutivas	$O(\log_{B+1} N + k/B)$

Insertar y eliminar

Una búsqueda hacia abajo y, en el peor caso, una partición o fusión por nivel hacia arriba: un número constante de bloques por nivel.

Para buscar, $\log_{B+1} N$ es óptimo. ¿También para insertar?

No. Si las consultas pueden esperar, hay estructuras (*buffer trees*) que insertan en $O\left(\frac{1}{B} \log_{M/B} \frac{N}{B}\right)$ amortizado: menos de una transferencia por operación.

¿Y si ordenamos con un B-Tree?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

En el modelo RAM funciona

Insertar los N elementos en un árbol balanceado y recorrerlo en orden es un ordenamiento óptimo: $\Theta(N \lg N)$.

¿Y si ordenamos con un B-Tree?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

En el modelo RAM funciona

Insertar los N elementos en un árbol balanceado y recorrerlo en orden es un ordenamiento óptimo: $\Theta(N \lg N)$.

En memoria externa

Son N inserciones de $\log_{B+1} N$ transferencias: $O(N \log_{B+1} N)$. El ordenamiento por mezcla múltiple cuesta $O\left(\frac{N}{B} \log_{M/B} \frac{N}{B}\right)$.

¿Y si ordenamos con un B-Tree?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

En el modelo RAM funciona

Insertar los N elementos en un árbol balanceado y recorrerlo en orden es un ordenamiento óptimo: $\Theta(N \lg N)$.

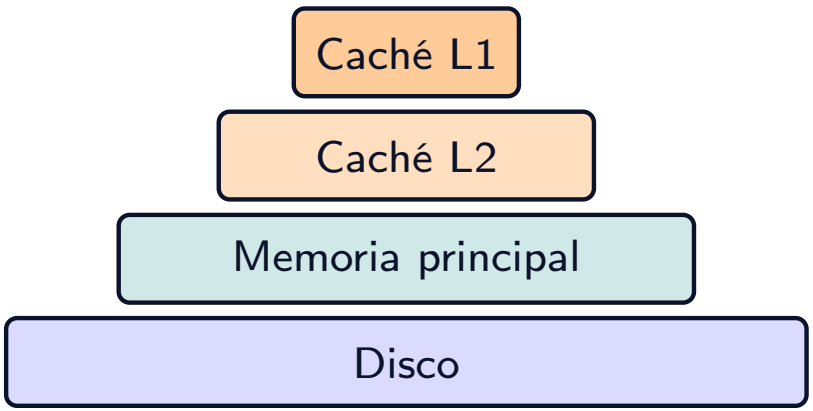
En memoria externa

Son N inserciones de $\log_{B+1} N$ transferencias: $O(N \log_{B+1} N)$. El ordenamiento por mezcla múltiple cuesta $O\left(\frac{N}{B} \log_{M/B} \frac{N}{B}\right)$.

Con $N = 10^{11}$, $M = 10^9$ y $B = 10^5$: unas $2 \cdot 10^{11}$ transferencias contra $2 \cdot 10^6$. El B-Tree responde cada operación *de inmediato*, y eso cuesta un factor B frente a procesar todo por lotes.

El modelo cache-oblivious

¿Cuál es el B?



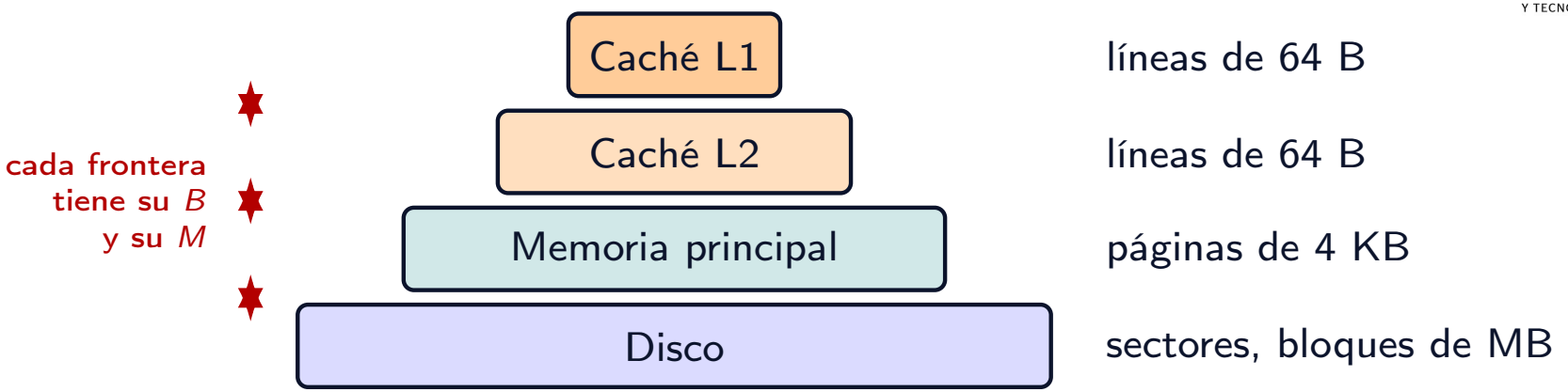
líneas de 64 B

líneas de 64 B

páginas de 4 KB

sectores, bloques de MB

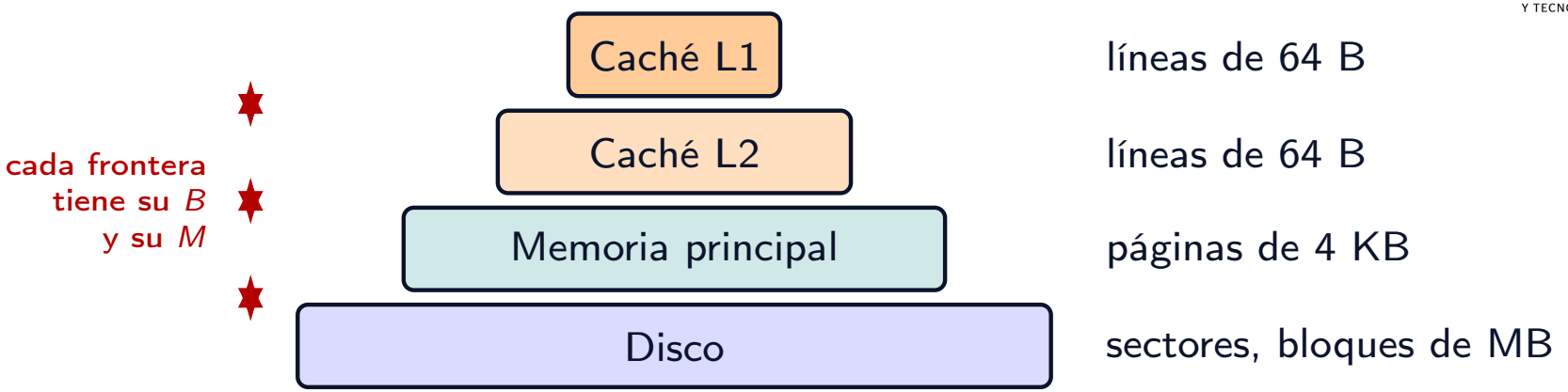
¿Cuál es el B?



El modelo de memoria externa mira **una** frontera

Un B-Tree con nodos del tamaño de una página de disco es óptimo entre RAM y disco, pero dentro de la RAM cada nodo son decenas de líneas de caché, recorridas sin ningún cuidado.

¿Cuál es el B?

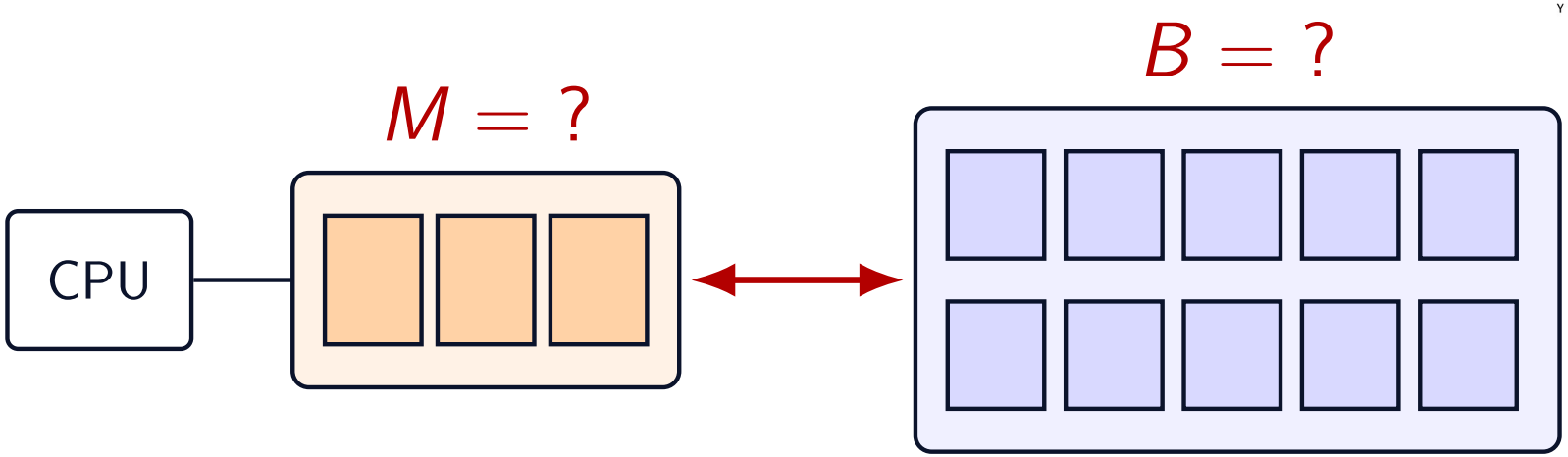


El modelo de memoria externa mira **una** frontera

Un B-Tree con nodos del tamaño de una página de disco es óptimo entre RAM y disco, pero dentro de la RAM cada nodo son decenas de líneas de caché, recorridas sin ningún cuidado.

¿Se puede ser eficiente en todas las fronteras a la vez, sin ajustar el algoritmo a cada máquina?

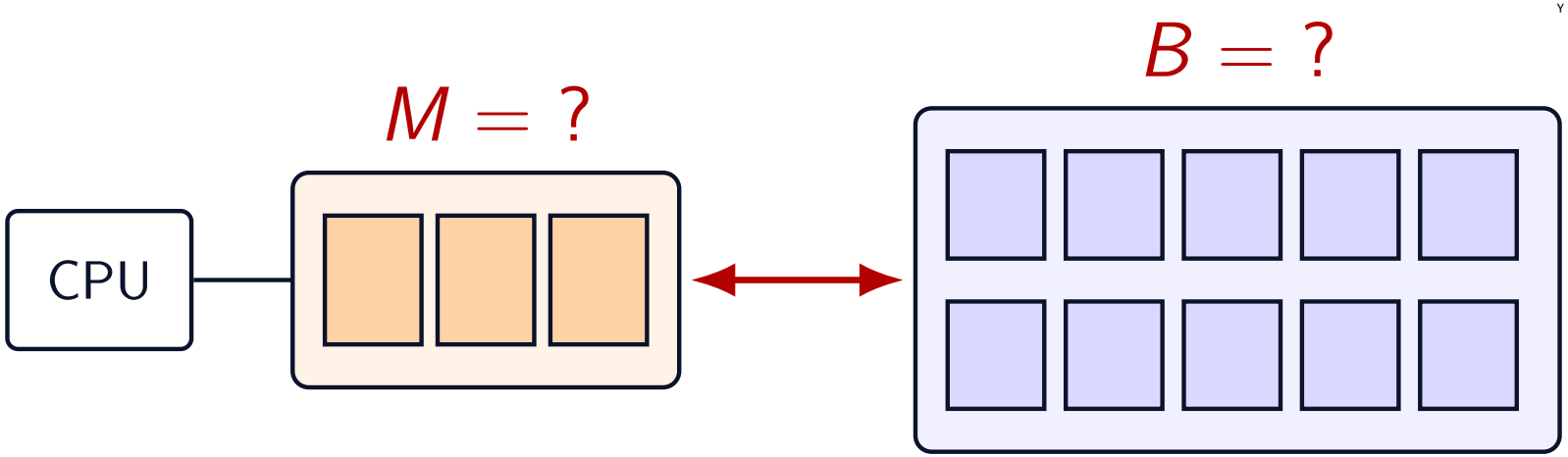
El modelo cache-oblivious



Frigo, Leiserson, Prokop y Ramachandran (1999)

El mismo modelo de dos niveles, con una restricción: el algoritmo **no conoce** B ni M.

El modelo cache-oblivious



Frigo, Leiserson, Prokop y Ramachandran (1999)

El mismo modelo de dos niveles, con una restricción: el algoritmo **no conoce** B ni M.

Se escribe como en RAM, se analiza como en memoria externa

El código no menciona bloques. B y M aparecen solo en el análisis, y la cota debe valer para **todos** sus valores.

¿Quién decide qué bloque sale de la memoria?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

En memoria externa, el algoritmo

Sabe qué tiene en memoria y elige qué bloque traer y cuál desalojar. Un algoritmo que no conoce B no puede hacerlo.

¿Quién decide qué bloque sale de la memoria?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

En memoria externa, el algoritmo

Sabe qué tiene en memoria y elige qué bloque traer y cuál desalojar. Un algoritmo que no conoce B no puede hacerlo.

En el modelo cache-oblivious, el sistema

Al acceder a una palabra que no está en memoria, su bloque se trae automáticamente. Si la memoria está llena, se desaloja otro bloque. Suponemos que esa elección es la **óptima**.

¿Quién decide qué bloque sale de la memoria?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

En memoria externa, el algoritmo

Sabe qué tiene en memoria y elige qué bloque traer y cuál desalojar. Un algoritmo que no conoce B no puede hacerlo.

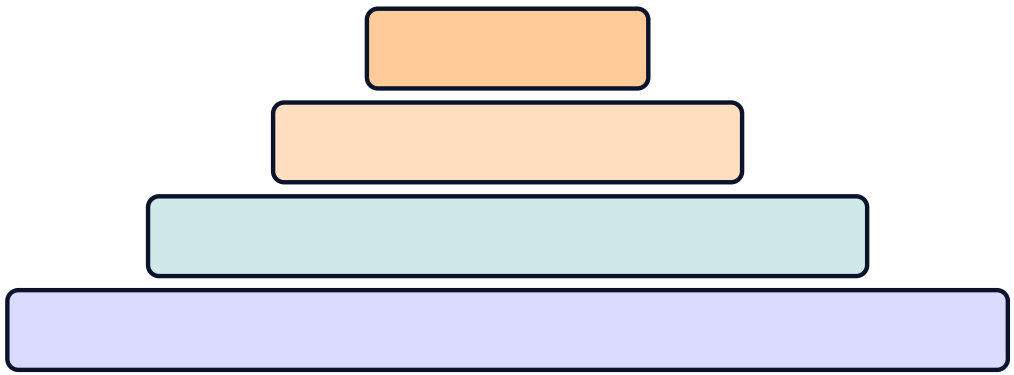
En el modelo cache-oblivious, el sistema

Al acceder a una palabra que no está en memoria, su bloque se trae automáticamente. Si la memoria está llena, se desaloja otro bloque. Suponemos que esa elección es la **óptima**.

¿No es mucho pedir que el sistema adivine el futuro?

No hace falta: las políticas reales LRU y FIFO, con una memoria del doble de tamaño, hacen a lo más el doble de transferencias que el óptimo. Y nuestras cotas no cambian al reemplazar M por $2M$.

¿Por qué es tan atractivo?

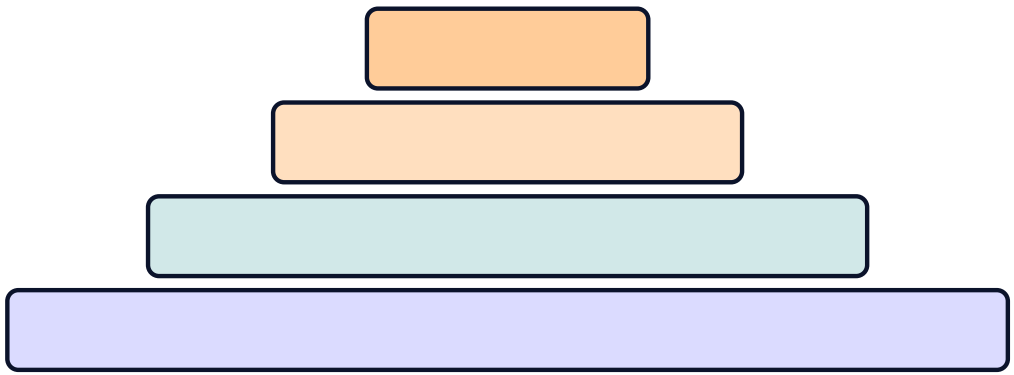


- B_1, M_1: óptimo
- B_2, M_2: óptimo
- B_3, M_3: óptimo

Una demostración, todas las fronteras

Si la cota vale para cualquier B y M, vale en particular para los de cada par de niveles vecinos: el algoritmo es bueno en **toda** la jerarquía a la vez.

¿Por qué es tan atractivo?



- B_1, M_1: óptimo
- B_2, M_2: óptimo
- B_3, M_3: óptimo

Una demostración, todas las fronteras

Si la cota vale para cualquier B y M, vale en particular para los de cada par de niveles vecinos: el algoritmo es bueno en **toda** la jerarquía a la vez.

Y además

No hay parámetros que ajustar al cambiar de máquina, y el algoritmo se adapta solo si la memoria disponible cambia mientras corre.

¿Qué sobrevive del tablero de memoria externa?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Problema	Memoria externa	Cache-oblivious
Escanear	$\Theta(N/B)$	Igual

Escanear

El algoritmo nunca usó B: recorre el arreglo en orden. B solo apareció al contar.

¿Qué sobrevive del tablero de memoria externa?

Problema	Memoria externa	Cache-oblivious
Escanear	$\Theta(N/B)$	Igual
Ordenar	$\Theta\left(\frac{N}{B}\log_{M/B}\frac{N}{B}\right)$	Igual, con $M = \Omega(B^{1+\varepsilon})$
Permutar	$\Theta\left(\text{mín}\{N,\ \textit{ordenar}\}\right)$	El mínimo es imposible

Escanear

El algoritmo nunca usó B: recorre el arreglo en orden. B solo apareció al contar.

Ordenar y permutar

La mezcla de M/B corridas necesita conocer M/B, pero hay otro algoritmo (*funnelsort*) que logra la misma cota. Para permutar, en cambio, no se puede saber cuál de las dos estrategias conviene.

¿Qué sobrevive del tablero de memoria externa?

Problema	Memoria externa	Cache-oblivious
Escanear	$\Theta(N/B)$	Igual
Ordenar	$\Theta\left(\frac{N}{B}\log_{M/B}\frac{N}{B}\right)$	Igual, con $M = \Omega(B^{1+\varepsilon})$
Permutar	$\Theta\left(\text{mín}\{N,\ \textit{ordenar}\}\right)$	El mínimo es imposible
Buscar	$\Theta(\log_{B+1} N)$	¿?

Escanear

El algoritmo nunca usó B: recorre el arreglo en orden. B solo apareció al contar.

Ordenar y permutar

La mezcla de M/B corridas necesita conocer M/B, pero hay otro algoritmo (*funnelsort*) que logra la misma cota. Para permutar, en cambio, no se puede saber cuál de las dos estrategias conviene.

¿Y buscar?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El B-Tree no sirve

Su definición empieza por "cada nodo guarda B claves". Sin conocer B, no se puede ni construir.

¿Y buscar?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El B-Tree no sirve

Su definición empieza por "cada nodo guarda B claves". Sin conocer B, no se puede ni construir.

Lo que sí podemos escribir sin conocer B

Un árbol binario de búsqueda balanceado, guardado en un arreglo. El algoritmo de búsqueda es el de siempre.

¿Y buscar?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El B-Tree no sirve

Su definición empieza por "cada nodo guarda B claves". Sin conocer B, no se puede ni construir.

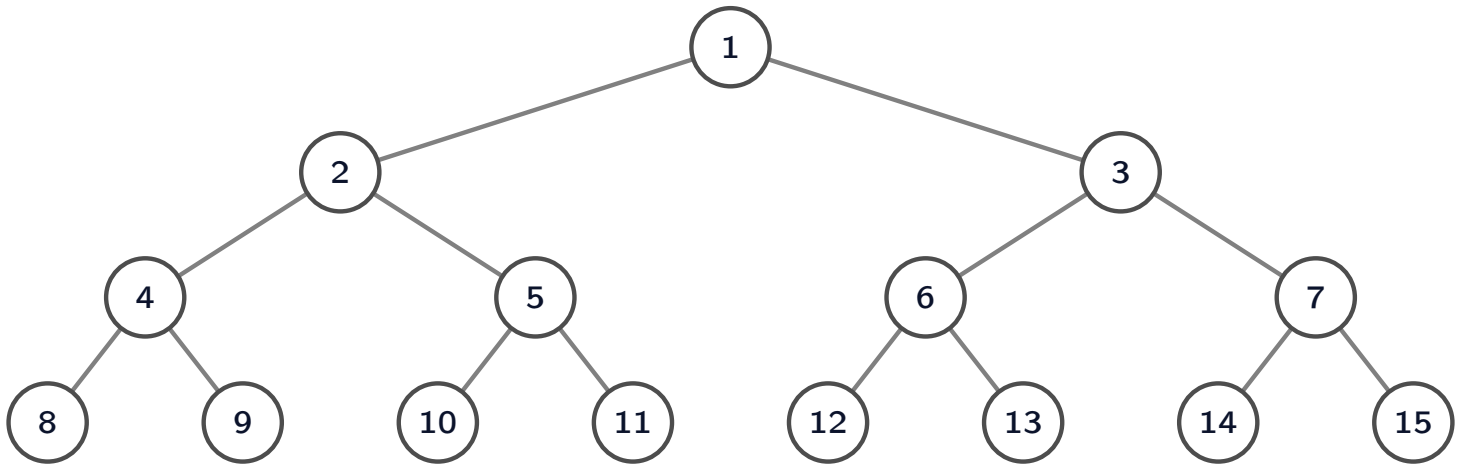
Lo que sí podemos escribir sin conocer B

Un árbol binario de búsqueda balanceado, guardado en un arreglo. El algoritmo de búsqueda es el de siempre.

Lo único que queda por decidir es en qué **orden** se guardan los nodos en el arreglo. ¿Importa?

Muchísimo: de ese orden depende qué nodos comparten bloque, y por lo tanto cuántos bloques toca un camino de la raíz a una hoja.

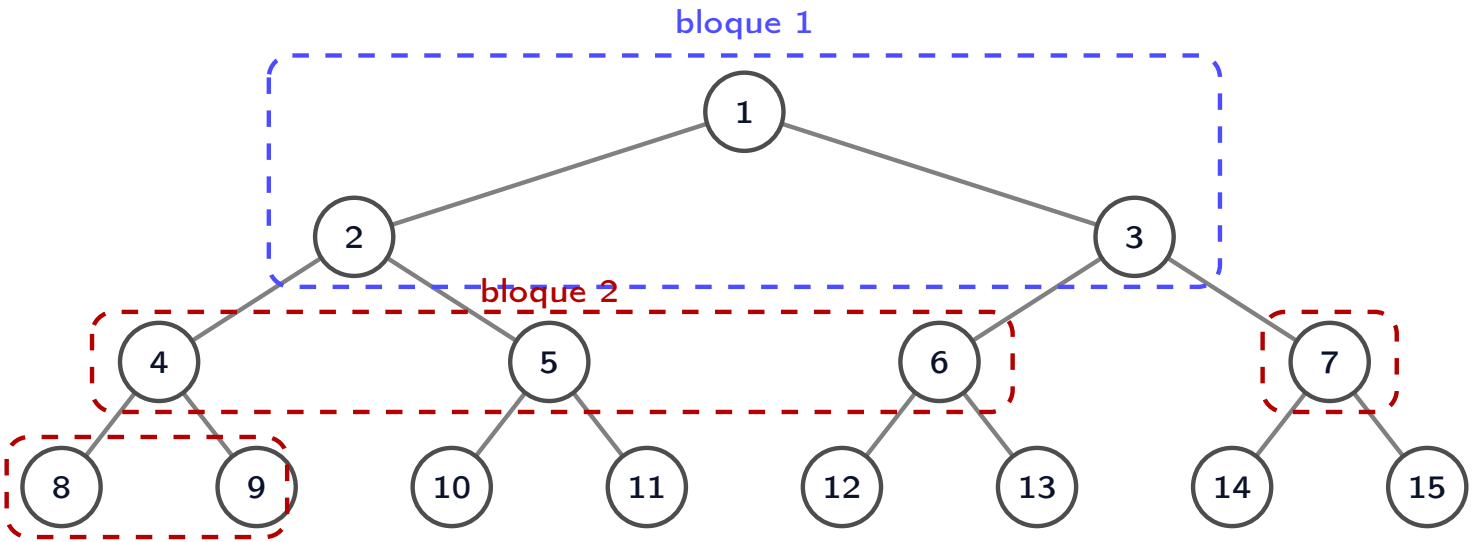
Primer intento: por niveles



El orden de un heap: $1;\ 2, 3;\ 4, 5, 6, 7;\ \ldots$

Los nodos se guardan nivel por nivel, de izquierda a derecha.

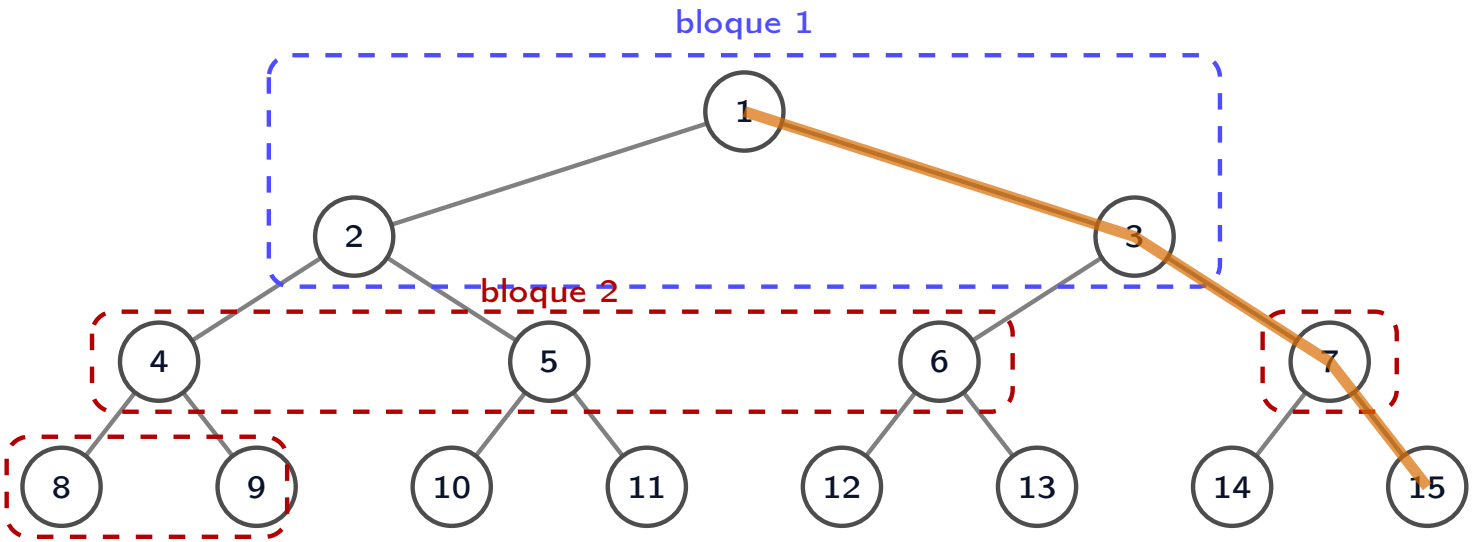
Primer intento: por niveles



El orden de un heap: $1;\ 2, 3;\ 4, 5, 6, 7;\ \ldots$

Con $B = 3$: la raíz y sus dos hijos comparten bloque. Pero el nivel siguiente ya se reparte entre varios.

Primer intento: por niveles



El orden de un heap: 1; 2, 3; 4, 5, 6, 7; ...

Los primeros $\lg B$ niveles salen casi gratis. Más abajo, un nodo y su hijo están a más de B posiciones de distancia: una transferencia por nivel. Total $\Theta(\lg N - \lg B)$, lo mismo que la búsqueda binaria.

Otros dos órdenes conocidos

En orden (el arreglo ordenado)

La búsqueda sobre este layout *es* la búsqueda binaria: $\Theta(\lg(N/B))$. Los últimos $\lg B$ pasos caen en un mismo bloque, pero los primeros saltan por todo el arreglo.

Otros dos órdenes conocidos

En orden (el arreglo ordenado)

La búsqueda sobre este layout *es* la búsqueda binaria: $\Theta(\lg(N/B))$. Los últimos $\lg B$ pasos caen en un mismo bloque, pero los primeros saltan por todo el arreglo.

En preorden

Bajar a la izquierda es pasar a la celda siguiente: barato. Bajar a la derecha salta por encima de todo el subárbol izquierdo. Un camino que siempre va a la derecha cuesta otra vez $\Theta(\lg(N/B))$.

Otros dos órdenes conocidos

En orden (el arreglo ordenado)

La búsqueda sobre este layout *es* la búsqueda binaria: $\Theta(\lg(N/B))$. Los últimos $\lg B$ pasos caen en un mismo bloque, pero los primeros saltan por todo el arreglo.

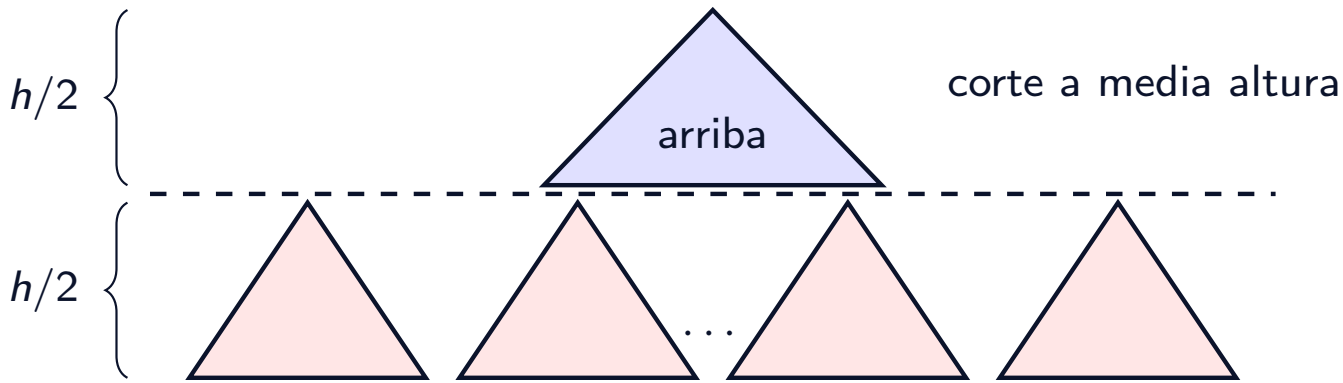
En preorden

Bajar a la izquierda es pasar a la celda siguiente: barato. Bajar a la derecha salta por encima de todo el subárbol izquierdo. Un camino que siempre va a la derecha cuesta otra vez $\Theta(\lg(N/B))$.

Los tres fallan por lo mismo

Agrupan bien los nodos en **una** escala (arriba, o abajo, o hacia un lado) y mal en las demás. Como no sabemos B, hay que agrupar bien en *todas* las escalas a la vez.

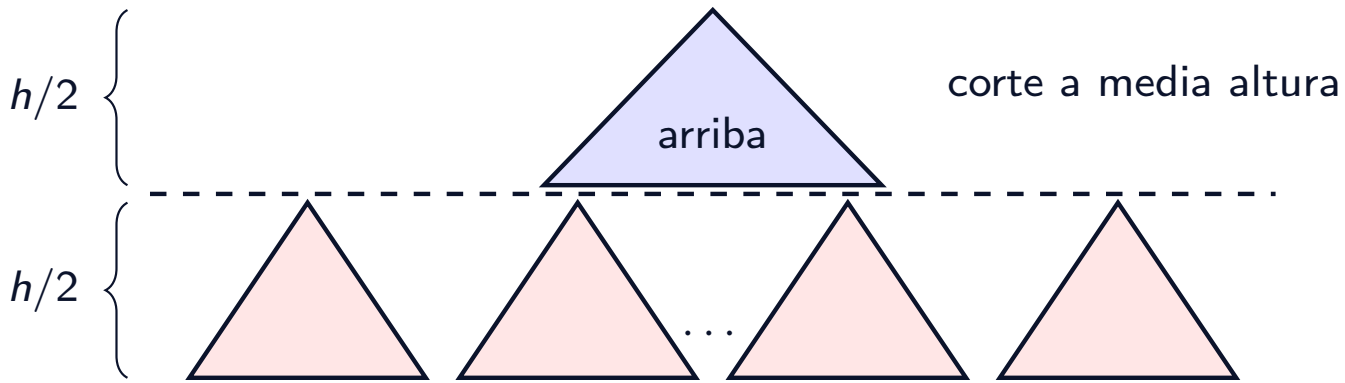
La idea que lo resuelve



Cortar el árbol a la mitad de su altura, recursivamente

Guardar primero la parte de arriba y luego cada subárbol de abajo, y hacer lo mismo dentro de cada pieza.

La idea que lo resuelve



Cortar el árbol a la mitad de su altura, recursivamente

Guardar primero la parte de arriba y luego cada subárbol de abajo, y hacer lo mismo dentro de cada pieza.

El layout de van Emde Boas

En cada nivel de la recursión las piezas son contiguas en memoria. Sea cual sea B, algún nivel de la recursión tiene piezas "del tamaño de un bloque". La próxima clase lo construimos y lo analizamos.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- B-Tree: nodos del tamaño de un bloque, entre $\lceil (B+1)/2 \rceil$ y $B+1$ hijos, todas las hojas al mismo nivel. Buscar, insertar y eliminar en $O(\log_{B+1} N)$ transferencias; para buscar, eso es óptimo.
- Crece partiendo nodos hacia arriba; se achica fusionándolos.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- B-Tree: nodos del tamaño de un bloque, entre $\lceil (B+1)/2 \rceil$ y $B+1$ hijos, todas las hojas al mismo nivel. Buscar, insertar y eliminar en $O(\log_{B+1} N)$ transferencias; para buscar, eso es óptimo.
- Crece partiendo nodos hacia arriba; se achica fusionándolos.
- Modelo cache-oblivious: el algoritmo no conoce B ni M; el sistema pagina por él. Una cota válida para todo B y M vale en toda la jerarquía.
- El orden en que se guarda un árbol en memoria decide su costo. Por niveles, en orden y en preorden cuestan $\Theta(\lg(N/B))$.

Logros de la sesión



Logros de la sesión

- Sabemos buscar, insertar y partir nodos en un B-Tree, y acotar su altura

Logros de la sesión

- Sabemos buscar, insertar y partir nodos en un B-Tree, y acotar su altura
- Sabemos explicar por qué un B-Tree no es la forma eficiente de ordenar en memoria externa

Logros de la sesión

- Sabemos buscar, insertar y partir nodos en un B-Tree, y acotar su altura
- Sabemos explicar por qué un B-Tree no es la forma eficiente de ordenar en memoria externa
- Sabemos definir el modelo cache-oblivious y justificar el supuesto de paginación óptima

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos buscar, insertar y partir nodos en un B-Tree, y acotar su altura
- Sabemos explicar por qué un B-Tree no es la forma eficiente de ordenar en memoria externa
- Sabemos definir el modelo cache-oblivious y justificar el supuesto de paginación óptima
- Sabemos analizar cuántas transferencias cuesta una búsqueda según el layout del árbol

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias

Referencias

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Bibliografía citada

- Bayer, R., & McCreight, E. (1972). Organization and maintenance of large ordered indexes. *Acta Informatica*, 1(3), 173–189.
- Frigo, M., Leiserson, C. E., Prokop, H., & Ramachandran, S. (1999). Cache-oblivious algorithms. *Proceedings of FOCS'99*, 285–298.
- Sleator, D. D., & Tarjan, R. E. (1985). Amortized efficiency of list update and paging rules. *Communications of the ACM*, 28(2), 202–208.
- Demaine, Erik (2021). *6.851: Advanced Data Structures (Spring'21)*. `https://courses.csail.mit.edu/6.851/spring21/`.

Agradecimientos

Estas diapositivas están basadas en las clases de Luciano A. Romero Calla para este curso (a su vez basadas en las clases de Erik Demaine, MIT 6.851), así como en clases similares de otras universidades.
Lista de créditos: Luciano A. Romero Calla, Erik Demaine, Keith Schwarz.